

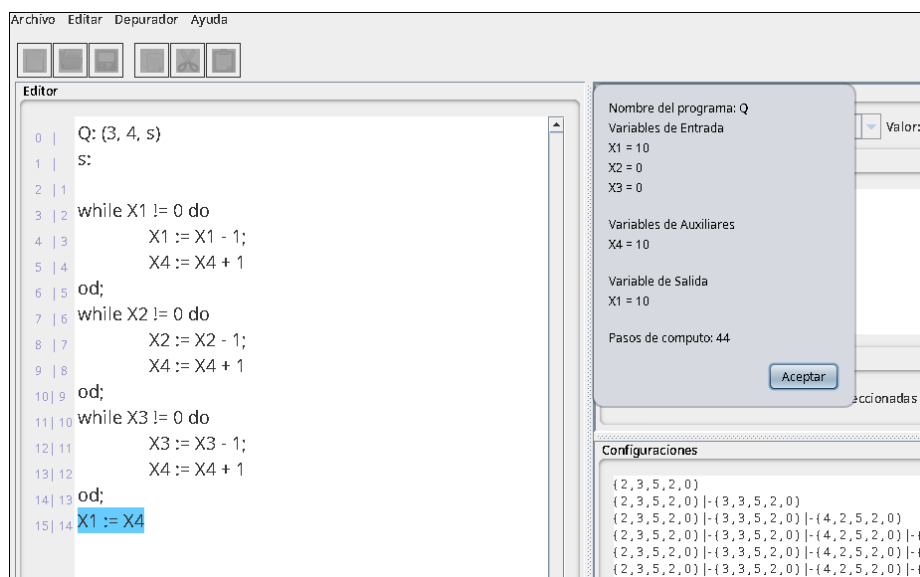
Práctica 4: WHILE y codificación

Salma Boulagna Moreno

Teoría de Autómatas y Lenguajes Formales

2 de junio de 2026

1. Actividad 1: suma de tres valores en WHILE



1.1. Idea de diseño

Se pide escribir un programa WHILE que calcule la suma de tres valores naturales. La función que se quiere obtener es:

$$F_Q(x_1, x_2, x_3) = x_1 + x_2 + x_3.$$

Para hacerlo se utiliza una variable auxiliar, X_4 , como acumulador. La idea es recorrer las tres variables de entrada y, mientras cada una sea distinta de cero, restarle una unidad y sumar una unidad en X_4 . Cuando los tres bucles han terminado, el valor acumulado se copia en X_1 , ya que la función calculada por un programa WHILE se lee en la primera variable.

1.2. Programa usado en Python

El programa se ha escrito como una cadena para poder ejecutarlo con la librería `whilelanguage`:

```
from python.whilelanguage import *
from python.whilelanguage.encoding import *

S = """(3,
while X1!=0do
  X1:=X1-1;
  X4:=X4+1;
od;
while X2!=0do
  X2:=X2-1;
  X4:=X4+1;
od;
while X3!=0do
  X3:=X3-1;
  X4:=X4+1;
od;
X1:=X4)
```

```

X4:=X4+1
od;
while X2!=0 do
X2:=X2-1;
X4:=X4+1
od;
while X3!=0 do
X3:=X3-1;
X4:=X4+1
od;
X1:=X4)"""

print("tq", t_steps(S, [3, 5, 2]))
print("calq:", f_function(S, [3, 5, 2]))

```

1.3. Ejecución y resultado

La entrada comprobada es (3, 5, 2), por lo que el valor esperado es:

$$3 + 5 + 2 = 10.$$

La ejecución devuelve:

```

tq 44
calq: 10

```

Por tanto:

$$F_Q(3, 5, 2) = 10, \quad T_Q(3, 5, 2) = 44.$$

El resultado final coincide con la suma esperada. Además, el valor de `t_steps` indica que el programa tarda 44 pasos en alcanzar la configuración final para esa entrada.

2. Actividad 2: programa que calcula la función diverge

2.1. Definición del programa

La función **diverge** es una función sin argumentos que no termina nunca. Como las variables del programa empiezan inicializadas a cero, basta con hacer que X_1 valga uno y después entrar en un bucle que nunca pueda finalizar.

El código WHILE elegido es:

```
X1:=X1+1; while X1!=0 do X1:=X1+1 od
```

En Python se guarda en la variable D:

```
D = "X1:=X1+1; while X1!=0 do X1:=X1+1 od"
```

2.2. Justificación

Al comenzar la ejecución se tiene:

$$X_1 = 0.$$

La primera instrucción hace que:

$$X_1 = 1.$$

A partir de ahí se evalúa el bucle:

```
while X1!=0 do X1:=X1+1 od
```

Como dentro del bucle X_1 aumenta en una unidad, nunca vuelve a valer cero. Por tanto, la condición $X_1 \neq 0$ sigue siendo verdadera indefinidamente y el programa no se detiene.

2.3. Codificación

La función `while2n` utilizada en la práctica recibe dos argumentos: el número de variables de entrada y el código del programa. Como `diverge` tiene cero argumentos, se codifica con:

```
print(while2n(0, D))
```

La salida obtenida es:

```
139273919255627
```

Así, la codificación del programa completo es:

$$\ulcorner(0, D)\urcorner = 139273919255627.$$

Si únicamente se codifican las instrucciones, sin añadir el número de variables de entrada, se obtiene con `code2n(D)`. En este caso:

```
code2n(D) = 16689751
```

3. Actividad 3: enumeración de vectores de $\mathbb{N}^*$

3.1. Idea de diseño

El conjunto $\mathbb{N}^*$ contiene todos los vectores finitos de naturales:

$$\mathbb{N}^* = \bigcup_{n \geq 0} \mathbb{N}^n.$$

En particular, contiene el vector vacío, vectores de longitud uno, vectores de longitud dos, etc. Para enumerarlos se utiliza la decodificación de Gödel implementada en la librería. La idea es recorrer los números naturales y aplicar `godeldecoding` a cada uno de ellos.

3.2. Función implementada

```
def primeros_vectores(k):
    salida = []

    for numero in range(k):
        salida.append(godeldecoding(numero))

    return salida

print(primeros_vectores(10))
```

3.3. Comprobación para $k = 10$

La ejecución para los diez primeros vectores produce:

```
[[], [0], [0, 0], [1], [0, 0, 0], [1, 0], [2],  
[0, 0, 0, 0], [1, 0, 0], [0, 1]]
```

3.4. Interpretación

La función recorre:

$$0, 1, 2, 3, \dots, k - 1$$

y decodifica cada número natural como un vector finito. Por eso, el orden de aparición de los vectores es el orden inducido por la codificación de Gödel usada por la librería.

Esto muestra que $\mathbb{N}^*$ es enumerable: existe un procedimiento efectivo que permite listar todos sus elementos.

4. Actividad 4: enumeración de programas WHILE

4.1. Idea de diseño

Los programas WHILE también se pueden codificar mediante números naturales. Por tanto, para enumerar programas basta con recorrer números naturales y decodificar cada uno con la función `n2while` de la librería.

4.2. Función implementada

```
def primeros_while(k):  
    listado = []  
  
    for codigo in range(k):  
        listado.append(n2while(codigo))  
  
    return listado  
  
lista = primeros_while(10)  
  
for posicion, programa in enumerate(lista):  
    print(posicion, programa)
```

4.3. Comprobación para $k = 10$

La salida obtenida es:

```
0 (0, X1:=0)  
1 (1, X1:=0)  
2 (0, X1:=0; X1:=0)  
3 (2, X1:=0)  
4 (1, X1:=0; X1:=0)  
5 (0, X1:=X1)  
6 (3, X1:=0)  
7 (2, X1:=0; X1:=0)
```

```
8 (1, X1:=X1)
9 (0, X1:=0; X1:=0; X1:=0)
```

4.4. Interpretación

Cada número natural se interpreta como la codificación de un programa WHILE. La función `n2while` realiza la decodificación y devuelve el programa correspondiente.

El procedimiento seguido es:

$$0 \mapsto n2while(0), \quad 1 \mapsto n2while(1), \quad 2 \mapsto n2while(2), \quad \dots$$

Por tanto, los programas WHILE forman un conjunto enumerable. Esta idea es importante porque permite tratar los programas como datos y asignar a cada programa un número natural.

5. Conclusión

En esta práctica se ha trabajado con programas WHILE, funciones parciales y codificaciones mediante números naturales. Primero se ha implementado un programa que calcula la suma de tres valores y se ha comprobado tanto su resultado como su tiempo de parada. Después se ha construido un programa que no termina, representando así la función `diverge`.

También se ha utilizado la librería de codificación para enumerar tanto los vectores de $\mathbb{N}^*$ como los programas WHILE. En ambos casos se usa la misma idea teórica: recorrer los números naturales y decodificar cada número según la codificación correspondiente.